juliacon

WENDy.jl

Nicholas Rummel[1], Stephen Becker[1], Daniel Messenger[2], Vanja Dukic[1], and David M. Bortz[1]

[1]University of Colorado Boulder
[2]Los Alamos National Laboratory

1. Summary

The Weak-form Estimation of Non-linear Dynamics (WENDy) framework is a recently developed approach for parameter estimation of systems of ordinary differential equations (ODEs). Prior work demonstrated WENDy to be robust, computationally efficient, and accurate, but only works for ODEs that are linear-in-parameters. All the algorithms in this package accommodate ODEs that are nonlinear-in-parameters. The WENDy-MLE algorithm in particular approximates a maximum likelihood estimator via local non-convex optimization methods; it has better accuracy, a substantially larger domain of convergence, and is often faster than other weak form methods and the conventional output error least squares method.
The code was designed to provide a mostly automated process; this facilitates an application friendly interface. We utilize symbolic programming to obtain an analytic expressions for the likelihood function and its first and second order derivatives. Moreover, unlike the prior algorithms which focus only on additive Gaussian noise, we extend the framework to accommodate data corrupted by either multiplicative log-normal noise or additive Gaussian noise. The WENDy.jl package efficiently and accessibly implements many weak form methods that were previously unavailable to the Julia community.

2. Statement of need

Julia has a bountiful ecosystem of numerical solvers that can directly solve ordinary differential equations [1; 11]. Furthermore, considerable effort has been made to facilitate automatic differentiation [12]. This supports downstream optimization routines that can be used for control, optimal design, and, more pertinently to this work, parameter estimation. However, in spite of these efficiencies, using forward simulation at every iteration of an optimization routine can rapidly increase the computational cost, especially in high-dimensional or stiff systems. For chaotic systems, using forward simulation also can introduce numerical instability. In contrast, weak form methods do not suffer in this way. Thus, to tackle more challenging systems, alternative approaches such as weak form methods become attractive. These methods do not rely on forward simulation of the differential equations to accomplish estimation. Instead, by optimizing the *equation error* rather than the *output error*, weak form methods efficiently optimize the parameters irrespectively of the challenge of forward simulation of the system.

3. Research impact statement

Interest in weak form methods have been revitalized in the last decade [7]. These methods are used in broader applications beyond parameter estimation; WSINDy uses sparse regularization to simultaneously identify the governing equations of ordinary differential equations [6] and partial differential equations [5] that are linear in parameters. Furthermore, WSINDy has been used in conjunction with dimensionality reduction techniques to effectively identify reduced order models [14]. WENDy.jl brings many existing weak form methods [2] into the Julia ecosystem as well as integrates novel approaches for systems that are non-linear in parameters and have an error structure beyond simple additive *iid* Gaussian noise. Specifically, in this work we construct an analytic expression for an approximation of a likelihood function, build Julia functions for the likelihood, its first, and its second order derivatives with symbolic programming, and then perform second order optimization to estimate parameters. All algorithms were extended to accommodate systems of ordinary differential equations that are both linear or nonlinear in parameters. In the linear in parameters case, we leverage computational advantages where available. Also, the code effectively handles data that has been corrupted by both additive Gaussian and multiplicative LogNormal noise.

4. State of the field

The underlying purpose of our approach is to obtain an estimator for unknown parameters $\mathbf{p} \in \mathbb{R}^J$ in an ordinary differential equation of the form

$$\dot{\boldsymbol{u}}(t) = \mathbf{f}(\boldsymbol{u}, t, \mathbf{p}). \tag{1}$$

Here, the right hand side function $\mathbf{f} : \mathbb{R}^D \times \mathbb{R} \times \mathbb{R}^J \to \mathbb{R}^D$ is defined on a compact domain $t \in [0, T]$ for state variable $\boldsymbol{u} \in \mathbb{R}^D$, and may be linear or nonlinear in t, $\boldsymbol{u}$, and $\mathbf{p}$.
The weak form of a differential equation is obtained by taking the inner product on both side of the equation with an arbitrary test function φ along each dimension

$$-\langle \dot{\varphi}, \boldsymbol{u} \rangle = \langle \varphi, \mathbf{f}(\boldsymbol{u}, \mathbf{t}, \mathbf{p}) \rangle. \tag{2}$$

Given data on a uniform grid $\left\{(\mathrm{t}_m, \mathrm{u}_m)\right\}_{m=0}^{M}$ and selecting specific test functions $\{\varphi\}_{k=1}^{K}$, the inner products can be approximated through quadrature. This can be written succinctly by introducing a data matrix $\mathbf{U} \in \mathbb{R}^{(M+1)\times D}$, a test function matrix $\boldsymbol{\Phi} \in \mathbb{R}^{K\times(M+1)}$, the matrix-valued function $\mathbf{F} : \mathbb{R}^{(M+1)\times D} \times \mathbb{R}^{M+1} \times \mathbb{R}^J \to \mathbb{R}^{(M+1)\times D}$ which naturally extends to $\mathbf{f}$, and a quadrature matrix $\mathbf{Q} \in \mathbb{R}^{(M+1)\times(M+1)}$

$$-\dot{\boldsymbol{\Phi}}\mathbf{Q}\mathbf{U} \approx \boldsymbol{\Phi}\mathbf{Q}\mathbf{F}(\mathbf{U}, \mathbf{t}, \mathbf{p}). \tag{3}$$

Moving all the parameters to the right hand side, we can define an expression for the *weak residual*

$$\mathbf{r}(\mathbf{p}) := \text{vec}\left[\dot{\mathbf{\Phi}}\mathbf{Q}\mathbf{F}(\mathbf{U}, \mathbf{t}, \mathbf{p}) + \mathbf{\Phi}\mathbf{Q}\mathbf{U}\right] \in \mathbb{R}^{KD} \tag{4}$$

where vec vectorizes the matrix expression on the right hand side of eq. (3). Because the quadrature *approximates* the underlying integrals, and the data has been *corrupted by noise*, one can expect r to be *small*, albeit not zero, when evaluated at the true parameters. By leveraging a known distribution of additive Gaussian noise and a linearization, one can obtain that the scaled weak residual is approximately distributed as a Gaussian

$$\mathbf{S}(\mathbf{p})^{-1/2}\mathbf{r}(\mathbf{p}) \overset{approx}{\sim} \mathcal{N}(0, I). \tag{5}$$

where the specific form of the covariance matrix $\mathbf{S}$ and more details can be found in [13]. A similar expression is obtained in the case of multiplicative Log-Normal noise. The WENDy-MLE algorithm leverages eq. (5) to obtain the approximate maximum likelihood estimator (A-MLE) via optimization

$$\hat{\mathbf{p}} = \underset{\mathbf{p}}{\text{argmin}}\Big(\text{logdet}\big(\mathbf{S}(\mathbf{p})\big) + \mathbf{r}(\mathbf{p})^\top \mathbf{S}(\mathbf{p})^{-1}\mathbf{r}(\mathbf{p})\Big). \tag{6}$$

Julia makes it possible to obtain first and second order derivatives of the objective function prior to performing optimization through symbolic computations and our custom implementation. This allows for efficient use of second order routines such as Trust Region methods.

5. Software design

WENDy.jl [13] is the first implementation of these weak form methods in Julia. The package utilizes other Julia packages such as Tullio.jl [8], Symbolics.jl [3], Optim.jl [4], JSOSolvers.jl [9], and NonlinearSolve.jl [10] to aid in custom implementation of novel mathematical algorithms. The code was designed so that users can simply define a struct

```
prob = WENDyProblem(
    tt::AbstractVector{<:Real},
    U::AbstractVecOrMat{<:Real},
    f::Function,
    J::Int
)
```

The only required inputs are `tt`, `U`, `f`, and `J` which correspond to the vector of time points for the observation data, a matrix whose columns contain the observation data, the function that defines the differential equation of interest, and the number of parameters the user wishes to estimate, respectively. There are optional parameters that allow the user to specify more information, see our documentation for details.
The WENDy.jl algorithms then build the necessary parameter estimation problem by first creating the necessary test function matrix, computing symbolic derivatives, and then building the Julia functions necessary to perform optimization. This all happens when the `WENDyProblem` is constructed. Then, the user can solve the parameter estimation problem by calling

```
pHat = solve(
    wendyProb::WENDyProblem,
    p0::AbstractVector{<:Real}
)
```

The user is expected to provide an initial guess `p0` for the parameters of interest.
We focused on providing users with a simple software interface so the weak form algorithms can be used with as little expert knowledge as possible, with the hope that this will result in a broader appeal for weak form methods use in applications.

6. Short Demonstration

Consider the Logistic Growth equation

$$\dot{u} = p_1 u - p_2 u^2, t \in [0, T]. \tag{7}$$

Data is observed on a uniform grid $\{(m\Delta t, \mathrm{u}_m)\}_{m=0}^{M}$, the observations are corrupted by multiplicative noise

$$\mathrm{u}_m = u(t_m)\eta_m, \ \forall m \in \{0, 1, \cdots, M\}$$

where $\log(\eta_m) \overset{iid}{\sim} N(0, \rho^2)$, and the true parameters are $\mathbf{p}^* = [2.25, 7.0]^\top$. For the experiment shown below, the hyperparameters are set to $T = 10, M = 100, \Delta t = 1/10, \rho = 1/5$. One instance of the data, the estimated trajectory, and the true trajectory is plotted in fig. 1.

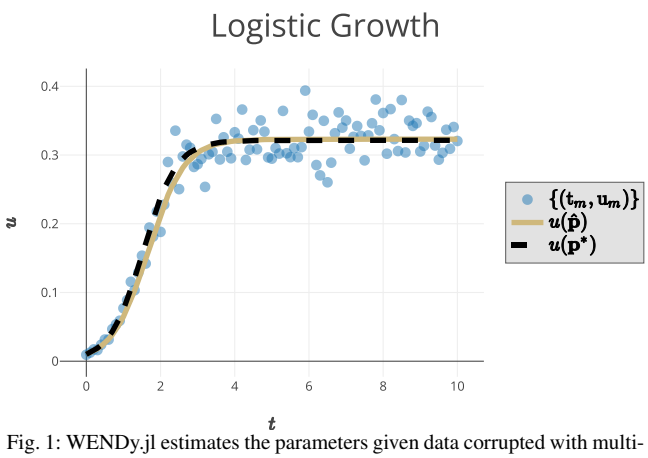

Fig. 1: WENDy.jl estimates the parameters given data corrupted with multiplicative noise. The observation data is shown with blue markers, the state's trajectory with the estimated parameters is shown with gold line, and the state's trajectory with the true parameters is shown with the dashed black line.

WENDy.jl obtains the A-MLE from an initial guess of $\mathbf{p}_0 = [2.93, 3.53]^\top$ and performing optimization of eq. (6). In this example, our estimated parameters have a coefficient relative error of 8.9% and a relative average forward solve error of 3%. As a quick point of reference, a standard output error least squares algorithm resulted in a coefficient relative error 11% and a relative average forward solve error of 3.5%. To generate these results, see *juliaCon2026_extendAbstractResults.jl* in the Git repository. For more extensive examples, comparisons, and numerical results see [13].

7. AI usage disclosure

No generative AI tools were used in the development of this software, the writing of this manuscript, or the preparation of supporting materials.

8. Acknowledgements

We acknowledge contributions from Jackson Krebsbach (University of Colorado), Will Houser (University of Colorado), April Tran (University of Colorado), and Lloyd Fung (Imperial College) for insights regarding software development for weak form methods and mathematical rigor.
This work is supported in part by the National Institute of General Medical Sciences grant R35GM149335, National Science Foundation grant 2109774, National Institute of Food and Agriculture grant 2019-67014-29919, and by the Department of Energy, Office of Science, Advanced Scientific Computing Research under Award Number DE-SC0023346.

References

[1] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A Fresh Approach to Numerical Computing. *SIAM Rev.*, 59(1):65–98, January 2017. doi:10.1137/141000671.

[2] David M. Bortz, Daniel A. Messenger, and Vanja Dukic. Direct Estimation of Parameters in ODE Models Using WENDy: Weak-form Estimation of Nonlinear Dynamics. *Bulletin of Mathematical Biology*, 85(110), 2023. doi:10.1007/S11538-023-01208-6.

[3] Shashi Gowda, Yingbo Ma, Alessandro Cheli, Maja Gwóźzdź, Viral B. Shah, Alan Edelman, and Christopher Rackauckas. High-performance symbolic-numerics via multiple dispatch. *ACM Communications in Computer Algebra*, 55(3):92–96, September 2021. doi:10.1145/3511528.3511535.

[4] Patrick K Mogensen and Asbjørn N Riseth. Optim: A mathematical optimization package for Julia. *JOSS*, 3(24):615, April 2018. doi:10.21105/joss.00615.

[5] Daniel A. Messenger and David M. Bortz. Weak SINDy For Partial Differential Equations. *Journal of Computational Physics*, 443:110525, October 2021. doi:10.1016/j.jcp.2021.110525.

[6] Daniel A. Messenger and David M. Bortz. Weak SINDy: Galerkin-Based Data-Driven Model Selection. *Multiscale Modeling & Simulation*, 19(3):1474–1497, 2021. doi:10.1137/20M1343166.

[7] Daniel A. Messenger, April Tran, Vanja Dukic, and David M. Bortz. The Weak Form Is Stronger Than You Think. *SIAM News*, 57(8), October 2024.

[8] Abbott Michael. Tullio.jl, February 2026.

[9] Tangi Migot, Dominique Orban, and Abel Soares Siqueira. JSOSolvers.jl: JuliaSmoothOptimizers optimization solvers. Zenodo, October 2024. doi:10.5281/ZENODO.3991143.

[10] Avik Pal, Flemming Holtorf, Axel Larsson, Torkel Loman, Utkarsh, Frank Schäefer, Qingyu Qu, Alan Edelman, and Chris Rackauckas. NonlinearSolve.jl: High-Performance and Robust Solvers for Systems of Nonlinear Equations in Julia. *arXiv:2403.16341*, March 2024. 2403.16341.

[11] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia. *JORS*, 5(1):15, May 2017. doi:10.5334/jors.151.

[12] Jarrett Revels, Miles Lubin, and Theodore Papamarkou. Forward-Mode Automatic Differentiation in Julia. *arXiv:1607.07892*, July 2016. 1607.07892.

[13] Nic Rummel, Daniel A. Messenger, Stephen Becker, Vanja Dukic, and David M. Bortz. WENDy for Nonlinear-in-Parameters ODEs, October 2025. doi:10.48550/arXiv.2502.08881. 2502.08881.

[14] April Tran, Xiaolong He, Daniel A. Messenger, Youngsoo Choi, and David M. Bortz. Weak-Form Latent Space Dynamics Identification. *Computer Methods in Applied Mechanics and Engineering*, 427:116998, July 2024. doi:10.1016/j.cma.2024.116998.